

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»
Варіант №9

Виконав: студент групи AI-244
Косенко Владислав

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	356	5
Час, с	1.62	0.02
Токенів/с	225.6	352.6

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1242,8	195,6
Час, с	5,47	0,66

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь:

Усі додаткові токени пішли на генерацію внутрішнього ланцюжка думок (поле `thinking`), де модель аналізувала зміст запиту, перевіряла альтернативні міста та обирала мову написання. Інструктивна модель одразу прогнозує цільовий токен без проміжної чернетки роздумів.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь:

Міркувальні моделі демонструють значну перевагу в логічних, математичних та алгоритмічних задачах, де правильний результат вимагає покрокових обчислень (як у задачі про ручку й зошит). Їх використання є економічно й часово невиправданим на простих фактологічних запитах, класифікації та стисненні тексту, де роздуми лише збільшують затримку й витрати токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь:

Інструктивні моделі навчаються переважно методом SFT (Supervised Fine-Tuning) та вирівнювання за уподобаннями людини (RLHF/DPO) одразу видавати фінальну відповідь. Міркувальні моделі додатково тренуються через навчання з підкріпленням (RL) з винагородою за верифіковані кроки розв'язання, стимулюючи генерацію розгорнутих «чернеток» перед остаточним виводом.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь:

Ні, більша кількість токенів свідчить лише про більший обсяг виконаної текстуальної роботи або наявність роздумів, але не про архітектурну перевагу чи глибину знань. Модель може генерувати зайву вербалізацію («балакучість») без покращення якості або навіть заплутати себе під час тривалого виводу.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь:

Різниця у швидкості зумовлена архітектурою моделей, розміром контекстного вікна та кількістю активних параметрів: llama3.2:3b має 3 млрд параметрів, тоді як qwen3:4b — 4 млрд, що вимагає більшого обсягу обчислень матричних множень на кожен окремий токен. Крім того, швидкість залежить від довжини вже згенерованого контексту (KV-cache), обробка якого сповільнює інференс при довгих відповідях.

Отже міркувальні моделі слід застосовувати в ролі «аналітичних модулів» для розв'язання складних багатоетапних або математичних підзадач, де критична безпомилковість виводу. Інструктивні моделі є оптимальними для інтерактивних

інтерфейсів, маршрутизації та типових запитів користувачів завдяки мінімальній затримці та ощадливому використанню ресурсів.

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат практично однаковий, проте витрати за кількістю токенів відрізняються у 71,2 раза (356 проти 5 токенів), а за часом — у 81 раз (1,62 с проти 0,02 с). На задачі про ручку й зошит ситуація протилежна — саме розписаний хід думок забезпечив 100% точність (5 із 5 правильних) у qwen3:4b, тоді як інструктивна llama3.2:3b без міркувань дала лише 2 правильні відповіді з 5. Практичний висновок: тип моделі треба обирати під тип задачі, а для ланцюжка агента з 8 послідовними викликами різниця складе приблизно 43,8 с проти 5,3 с, що робить міркувальну модель занадто повільною для швидкого інтерактивного налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	273	Українська (зі суржилом та латиницею)	3	Частково
llama3.2:3b	українською, 2 речення	195	Змішана (суржик, вкраплення англійських слів)	2	Частково
qwen3:4b	лаконічний асистент	125	Українська (чиста, природна)	2	Так
qwen3:4b	українською, 2 речення	130	Українська (чиста, природна)	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	90	140
llama3.2:3b	українською, 2 речення	54	53	107
qwen3:4b	лаконічний асистент	55	1407	1462
qwen3:4b	українською, 2 речення	64	763	827

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь:

Обидві моделі формально вклалися у два речення при прямій вказівці, проте qwen3:4b дотрималась цієї лаконічності в обох експериментах і зробила це значно якісніше за змістом.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь:

Так, модель Llama3.2:3b змішувала українські слова з англійськими («certain», «contrary», «Relations») через малий обсяг українськомовного корпусу при її навчанні.

3. Як змінився prompt_tokens при зміні systemповідомлення і чому?

Відповідь:

Показник prompt_tokens зріс для обох моделей (з 50 до 54 у Llama та з 55 до 64 у Qwen). Це зумовлено тим, що друга системна інструкція містить довшу фразу («Відповідай лише українською, максимум двома реченнями»), яка потребує більше токенів для кодування токенизатором.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у Llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь:

Оскільки qwen3:4b є міркувальною моделлю, лічильник completion_tokens враховує як фінальну видиму відповідь, так і всі токени внутрішнього монологу (thinking), приховані від користувача.

Відповіді на 5 питання чат-боту Gemini (крок 7)

1. Я порівнював інструктивну модель (Llama3.2:3b) та міркувальну (qwen3:4b) з різними systemповідомленнями. Чому моделі порізно дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь:

Це залежить від розміру моделі, токенизатора та якості етапу Supervised Fine-Tuning (SFT) на дотримання інструкцій (instruction following). Менші моделі гірше тримають фокус на кількох обмеженнях одночасно через меншу кількість параметрів уваги, тоді як міркувальні моделі перед відповіддю детально планують дотримання правил у чернетці думок.

2. Чим system повідомлення відрізняється від userповідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь:

Архітектурно це просто текст із різними службовими префіксами (спеціальними токенами розмітки чату). Моделі навчають надавати вищий пріоритет системним директивам під час налаштування поведінки, проте пріоритет не є абсолютним і може порушуватися через конфлікти в промтах або джейлбрейки.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь:

Витрати вихідних токенів агрегують повну довжину генерації на етапі декодування. У міркувальної моделі сотні або тисячі токенів генеруються для ланцюжка думок (Chain-of-Thought), які потім просто не показуються у фінальному полі content.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь:

Поведінка стає недетермінованою: модель може або пріоритезувати системне правило завдяки тренуванню, або виконати інструкцію користувача через «recency bias» (свіжіший контекст у кінці послідовності). Малі моделі часто генерують гібридну або хаотичну відповідь, намагаючись догодити обом умовам одночасно.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь:

Малі моделі мають обмежену ємність пам'яті ваг і навчаються переважно на англійських датасетах, тому інші мовні розподіли у них слабкі. Під час генерації токенів токени сильнішої мови (англійської) мають вищу статистичну ймовірність, через що модель зривається на кальки чи англійські слова.

Отже, System-повідомлення — це базовий інструмент калібрування стилю, мови та контексту, проте воно не гарантує суворого дотримання меж, особливо в компактних моделях. Лише однієї текстової інструкції зазвичай недостатньо для критичних обмежень (як-от формат виводу або обсяг). Якщо модель ігнорує правила, слід переходити до структурованого виводу (JSON Schema / Pydantic), додавати кілька прикладів виконання (few-shot prompting) або реалізовувати валідацію на рівні бекенду з повторним запитом.

Висновок за завданням 2

Експеримент показав, що system-повідомлення суттєво змінює формат і поведінку моделі без потреби донавчання, однак чутливість до інструкцій сильно різниться між архітектурами. Інструктивна модель llama3.2:3b виявилася чутливою до системного промпу щодо довжини (скоротила відповідь з 273 до 195 символів), але через обмеженість навчальних даних припустилася мовного суржику («mathematicalний», «contrary»). Міркувальна qwen3:4b бездоганно дотрималась української термінології та обмеження у два речення, але ціною прихованої перевитрати ресурсів — 1407 та 763 токенів на міркування заради короткої відповіді. Практичний висновок: формулювання системної ролі є обов'язковим першим кроком налаштування, проте для малих інструктивних моделей потрібна додаткова фільтрація мови або використання міркувальних моделей для складного аналізу тексту.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.7-flash
Час відповіді, с	2.72	12.36
completion_tokens	143	172
Довжина відповіді, символів	504	501
Мова відповіді	Українська (граматично кострубата)	Українська (грамотна, природна)
Фактичних помилок у відповіді	2 (стверджує, що функція не має імені, а метод змінюється за час виконання)	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь:

У коді виклику не змінилося жодного рядка, оскільки використовується уніфікований інтерфейс OpenAI. Зміна звелася лише до вибору іншого ключа конфігураційного словника PROVIDERS (зміна значення змінної/аргументу у функції make_client).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь:

Хмарна модель Gemini надала значно точнішу й структурованішу відповідь. Вона чітко вказала головну ознаку — прив'язку методу до класу/об'єкта («кожен метод є функцією, але не кожна функція є методом»), підкріпивши це прикладами (print() проти car.drive()). Локальна ж модель припустилася грубої концептуальної помилки, заявивши, що «функція не має жодного імені».

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь:

Так, були і помилки в змісті (неправильні означення), і мовні дефекти — кострубата узгодженість слів («Функцію і методом розглядаються як різновидом логіки...»), «входних даних» замість «вхідних»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь:

У моєму випадку локальна виявилася швидшою: 2.72 с проти 12.36 с, це обумовлено тим що я запускаю локальну модель на сучасній GPU і модель повністю завантажена у VRAM.

Відповіді на 5 питання чат-боту Gemini (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь:

OpenAI першою випустила комерційно успішний API із простою та логічною схемою (messages, roles, completions). Підтримка цього стандарту стала де-факто необхідністю для індустрії, оскільки дозволяє розробникам перемикаати провайдерів та бекенди без зміни архітектури свого коду.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь:

Розмір моделі є критичним чинником для збереження фактичних знань, розуміння контексту та володіння різними мовами. Модель на 3B параметрів має фізично меншу місткість ваг і не здатна запам'ятати рідкісні мовні чи технічні зв'язки, що призводить до галюцинацій та мовних калюк, тоді як великі моделі мають вищі рівні узагальнення знань.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь:

Сервери дата-центрів паралелять генерацію на надшвидких кластерах GPU/TPU з високою пропускнуою здатністю пам'яті (до кількох ТБ/с HBM). Хоча мережевий пінг додає затримку до першого токена (TTFT), висока швидкість витоку наступних токенів дозволяє швидко компенсувати цей час на значних обсягах тексту.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь:

Передача даних стороннім хмарам несе ризик витоку конфіденційної інформації (РІІ, фінансові або медичні дані), а також використання промптів для донавчання моделей провайдера. У бізнес-проєктах це вирішують укладанням enterprise-угод із zero-data retention, попередньою анонімізацією чутливих сутностей (маскуванням) або розгортанням повністю локальних Open-Source LLM у закритому контурі компанії.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь:

Rate limit — це обмеження провайдера на кількість запитів за хвилину (RPM), день (RPD) або токенів за хвилину (TPM) для захисту від перевантажень. Розробники обходять ці бар'єри чергами запитів із затримками, експоненційним відкатом при помилці 429 (exponential backoff), кешуванням типових відповідей та ротацією кількох API-ключів чи провайдерів.

Висновок за завданням 3

Експеримент підтвердив практичну зручність використання стандарту OpenAI API: перемикання між локальним інференсом усередині Ollama та віддаленим сервісом Google AI Studio відбулося без зміни коду обробки даних завдяки єдиному клієнту. Порівняльний аналіз показав драматичну різницю в інтелектуальній спроможності: компактна локальна модель на 3 млрд параметрів

згенерувала фактичну помилку і показала слабе знання української термінології, тоді як потужна хмарна модель забезпечила бездоганну відповідь із логічним структуруванням та прикладами. Практичний висновок: єдина клієнтська абстракція дозволяє гнучко балансувати між дешевизною та автономністю локальних моделей на простих етапах і якістю хмарних моделей у критичних вузлах системи.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Під час виконання Завдання 4 при зверненні до хмарного API Google AI Studio виникала помилка 503 UNAVAILABLE (This model is currently experiencing high demand).

Щоб продовжити виконання роботи було реалізовано міграцію на хмарного провайдера — Groq (модель openai/gpt-oss-20b), передбаченого методичними вказівками.

Таблиця 7: Результати вимірювання швидкості

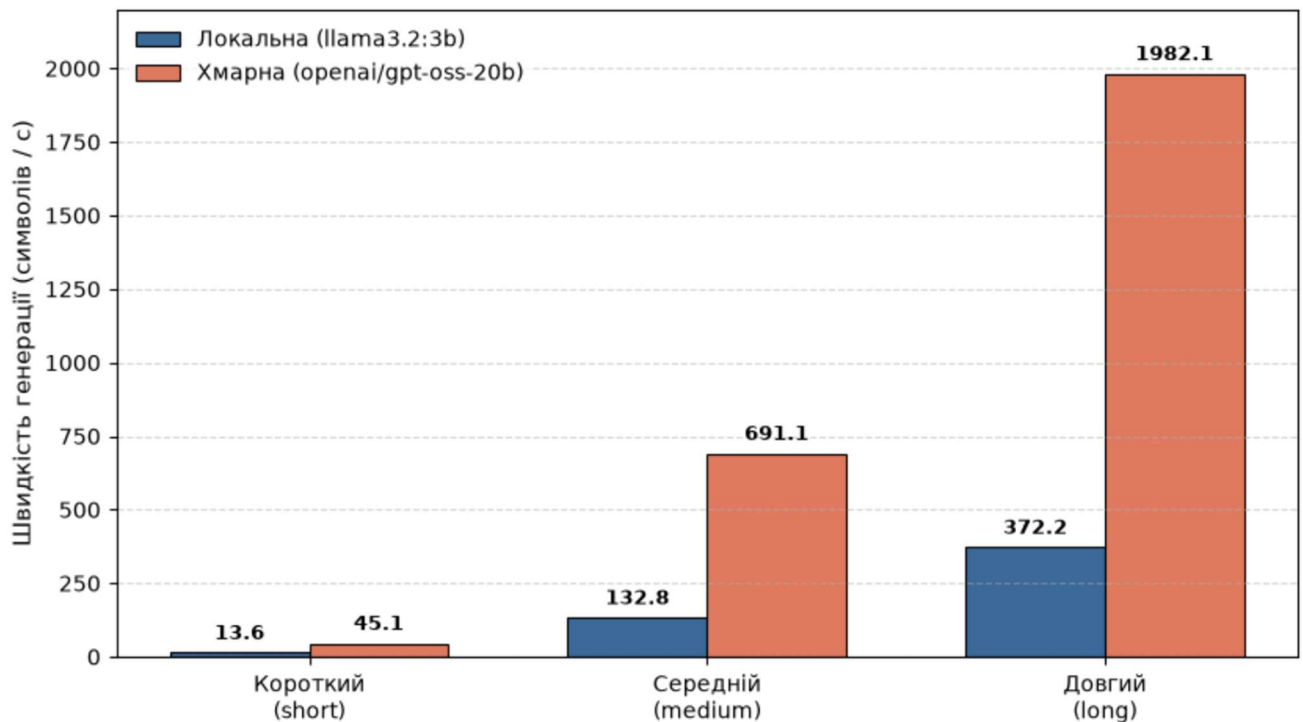
Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,057	2,127	13,6
локальна	llama3.2:3b	середній	2,041	2,469	132,8
локальна	llama3.2:3b	довгий	2,054	3,31	372,2
хмарна	gpt-oss-20b	короткий	0,547	0,548	45,1
хмарна	gpt-oss-20b	середній	0,507	0,669	691,1
хмарна	gpt-oss-20b	довгий	0,477	1,678	1982,1

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	3,794	2,095
Всього, с	3,846	2,28

Рисунок 1 - Стовпчикова діаграма (matplotlib) (крок 5)

Порівняння швидкості генерації: Локальна vs Хмарна модель (Варіант 9)



Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь:

Кращий показник TTFT продемонструвала хмарна модель openai/gpt-oss-20b (~0,48–0,55 с проти ~2,04–2,06 с у локальній), оскільки серверна інфраструктура використовує високопродуктивні тензорні прискорювачі та тримає ваги моделі постійно завантаженими у надшвидку VRAM.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь:

Так, у даному експерименті хмарна модель стала абсолютною переможницею за обома параметрами. Втім, на слабкому інтернет-з'єднанні TTFT хмари міг би просісти через мережевий пінг, навіть якби швидкість декодування токенів залишалася високою.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь:

Хмарна модель різко наростила відрив: якщо на короткому запиті вона випереджала локальну в 3,3 раза (45,1 проти 13,6 симв./с), то на довгому перевершила її більш ніж у 5,3 раза (1982,1 проти 372,2 симв./с).

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь:

Одного запуску недостатньо через неминучі системні флуктуації (коливання RTT у мережі, перемикання контексту ОС, фонові процеси). Три прогони з вибором медіани дозволяють повністю відсікти поодинокі аномальні викиди часу.

5. Чому холодний старт повільніший за теплий?

Відповідь:

При холодному старті Ollama додатково витрачає ~1,7 с на блокуюче читання ваг моделі з диска в пам'ять та ініціалізацію контексту, тоді як при теплому старті модель уже розгорнута й готова до розрахунків.

Відповіді на 5 питань чат-боту Gemini (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь:

Під час фази TTFT відбувається передача вхідного тексту мережею, токенизація промπτу та паралельний прорахунок механізму attention для всіх вхідних токенів (фаза prefill) з метою згенерувати найперший токен. Його вимірюють окремо від швидкості подальшого виводу, оскільки TTFT є критичною метрикою інтерактивності, що безпосередньо визначає суб'єктивне очікування людини перед початком читання.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь:

Хмарний запит обтяжений мережевим пінгом (RTT), SSL-рукостисканням, авторизацією та очікуванням у черзі API-шлюзу, що може збільшити початкову затримку. Проте подальший авторегресійний інференс виконується на кластерах із колосальною пропускною здатністю пам'яті (HBM), що дає багаторазово вищу швидкість генерації наступних токенів порівняно з локальним комп'ютером.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь:

Холодним стартом називають перший виклик моделі після запуску служби або команди вивантаження (`ollama stop`), коли її ваг ще немає в активній пам'яті. Сервер змушений виконувати повільні дискові операції для зчитування бінарного файлу моделі в RAM/VRAM, тоді як теплі запуски використовують уже кешовану структуру даних без накладних витрат I/O.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь:

Будь-який одиничний вимір піддається впливу випадкових факторів: стрибків затримки інтернет-пакетів, скидання системного дискового кешу чи завантаження процесора сторонніми службами. Медіана є статистично стійкою (робастною) оцінкою, яка, на відміну від середнього арифметичного, не спотворюється поодинокими критичними викидами.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь:

Вхідний промпт обробляється одночасно в рамках однієї високоефективної матричної операції множення (паралельний `prefill`). Генерація ж відповіді є авторегресійною: кожен наступний токен залежить від попереднього, через що модель змушена щоразу повторювати повний цикл проходу по всій сітці для кожного згенерованого слова окремо.

Отже, для інтерактивного чат-бота найважливішим є показник TTFT. Користувач сприймає систему як «живу» та чутливу, якщо перше слово з'являється менш ніж за 0,5–1 секунду, після чого стрімінг тексту дозволяє комфортно читати відповідь у міру її виведення. Для фонової пакетної обробки (batch processing) TTFT не відіграє майже жодної ролі, оскільки там відсутній живий оглядач. Критичною є виключно загальна швидкість генерації (симв./с або токенів/с), адже саме вона визначає кінцеву пропускну здатність конвеєра при аналізі тисяч сторінок тексту.

Висновок за завданням 4

Експериментальне вимірювання затримки підтвердило фундаментальну різницю між показником первинного відгуку (TTFT) та пропускну здатністю моделі (симв./с). Хмарна модель openai/gpt-oss-20b продемонструвала повну перевагу над локальною llama3.2:3b, забезпечивши TTFT на рівні ~0,5 с проти ~2,05 с та швидкість генерації до 1982,1 симв./с на довгих запитах. Дослідження стартів виявило, що холодний запуск локальної моделі сповільнює TTFT майже вдвічі (3,79 с проти 2,10 с) виключно через операції дискового завантаження ваг у пам'ять. Практичний висновок: метрика TTFT є вирішальною для побудови комфортних діалогових інтерфейсів реального часу, тоді як для об'ємних аналітичних пайплайнів критичною залишається чиста швидкість декодування токенів, у якій хмарні кластери багаторазово перевершують споживчі локальні конфігурації.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель openai/gpt-oss-20b
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	1	2
4. Код	0	1	1
5. Логіка	0	2	2
6. Українська	0	2	2
Разом / 12	4	10	11

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель openai/gpt-oss-20b
Факт	Ні	Ні	Ні
Формат	Ні	Ні	Ні
Стиснення	Ні	Ні	Ні
Код	Ні (обидва 0 балів)	Так(1-й: 2 бали, 2-й: 0 балів)	Ні
Логіка	Ні	Ні	Ні
Українська	Ні	Ні	Ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь:

Найбільше балів набрала хмарна модель (11 із 12), випередивши qwen3:4b (10) та розгромивши llama3.2:3b (4). Основна різниця сформувалася на завданнях

логічного міркування, роботі з числовим текстом та якості української мови, де компактна модель llama3.2:3b повністю провалилася.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь:

Розбіжність виникла у qwen3:4b на завданні 4 (генерація коду), де в першому запуску модель видала суто чистий код, а в другому додала розлогі пояснення й markdown-розмітку. Це свідчить про недетермінованість поведінки моделей за ненульової температури ($T = 0.7$) та нестабільність дотримання негативних інструкцій форматування («лише код»).

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь:

Так, усі три моделі в обох прогонах неочікувано впоралися з вимогою й повернули валідний однорядковий JSON без markdown-блоків.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь:

У моделі llama3.2:3b спостерігалася критична деградація мови із сумішню русизмів, вигаданих слів («вшкодується», «хоруд», «відріз») та англійських включень («chemical elements»). Моделі qwen3:4b та хмарна продемонстрували чисту літературну мову з коректною астрономічною термінологією.

Приклад провалу (крок 7)

Повний текст запиту:

«Зонд рухається зі швидкістю 60000 км/год, відстань становить 480 млн км. Скільки повних діб триватиме подорож? Покажи хід розв'язання.»

Повний текст відповіді (llama3.2:3b, прогін 1):

«ЗОНДРУХАПСЯ З ШОРОКОЮ ШЛЯБОМ!

1. Заданна швидкість зонда 60000 км/год.
2. Відстань 480 млн км.

3. Наявна швидкість земи 7368,8 км/год (приймова швидкість земної поверхні за 100 км/год).
4. Роздільна швидкість = швидкість зонда - швидкість земи = 60000 км/год - 7368,8 км/год = 52631,2 км/год
5. Время = відстань / роздільна швидкість = 480000000000 м/с = 4800000000000 м/год = 4800000000 км/год = 4800000000/52631,2 = 90691 діб»

Правильна відповідь мала бути:

$$\text{Час} = \frac{480\,000\,000 \text{ км}}{60\,000 \text{ км/год}} = 8000 \text{ годин}$$

$$8000/24 = 333,33 \text{ діб} \Rightarrow 333 \text{ повні доби.}$$

Пояснення, чому так сталося:

Модель llama3.2:3b має замалий обсяг параметрів (3B) для автономного багатоетапного міркування без CoT-підказок. Вона втратила зв'язок між фізичними величинами, згальюючи неіснуючу в умові «швидкість Землі», сплутала розмірності (метро-години) та провела абсолютно некоректні арифметичні дії через те, що autoregressive декодування малої моделі просто підбирало правдоподібні математичні символи замість реального обчислення.

Відповіді на 5 питання чат-боту Gemini (крок 8)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь:

На практиці використовують стандартизовані бенчмарки з автоматичними метриками (MMLU, GSM8K, HumanEval), оцінювання через сильніші моделі-судді (LLM-as-a-Judge, наприклад, через GPT-4), а також платформи краудсорсингового сліпого тестування з реальними користувачами (LMSYS Chatbot Arena).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь:

Під час етапу SFT (Supervised Fine-Tuning) та RLHF переважна більшість прикладів коду й структурованих даних у датасетах подавалася всередині тегів json Цей асоціативний патерн настільки міцно закріплений у вагах моделі, що системний промпт часто не здатен подолати високу ймовірність токена зворотної лапки.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь:

Розподіл навчального корпусу (pre-training data) містить менше 1% україномовних текстів порівняно з англійськими та російськими. Малий розмір моделі (3B) обмежує місткість пам'яті токенизатора й вагів, тому вона комбінує схожі граматичні шаблони з домінуючих слов'янських мов, утворюючи покручі та кальки.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь:

При параметрі температури вище нуля ($T > 0$) вибір наступного токена відбувається ймовірнісним семплюванням, а не жадібним вибором аргмаксимуму. Якщо на перших токенах випадково обирається альтернативна траєкторія, контекст кардинально змінюється, що може привести генерацію як до геніальної структури, так і до галюцинацій.

5. Що таке галюцинація мовної моделі і чому модель радше вигідає факт, ніж визнає незнання?

Відповідь:

Галюцинація — це статистична генерація поверхнево правдоподібного тексту, який не має під собою фактологічної основи. Модель прагне мінімізувати функцію втрат наступного ймовірного токена, і оскільки в навчальних текстах

люди рідко пишуть «я цього не знаю», модель відтворює домінантний патерн упевненої, змістовно заповненої відповіді.

Отже, оцінка за рубрикою є надійним інженерним фільтром, оскільки вона абстрагується від суб'єктивного сприйняття стилю та перевіряє конкретні крайові випадки: збереження чисел, стійкість до формату та відтворюваність логіки. Найбільш показовими виявилися завдання 5 (міркування) та 6 (мова), які одразу оголюють системну слабкість надмалих локальних моделей. Для повноцінної роботи з україномовним текстом варто обирати хмарні LLM або локальні моделі від 8B–14B параметрів із високою часткою багатомовного претрейну, тоді як моделі рівня Llama3.2:3b непридатні для аналітичних задач українською.

Висновок за завданням 5

Експеримент підтвердив ефективність інженерного оцінювання мовних моделей за стандартизованою рубрикою: використання заздалегідь визначених критеріїв і повторних прогонів усунуло суб'єктивне сприйняття «розумності» та дало чіткий профіль сильних і слабких сторін кожної моделі.

Порівняльний аналіз продемонстрував глибоку прірву між різними класами архітектур. Мала інструктивна модель Llama3.2:3b набрала лише 4 бали з 12: вона повністю провалила багатоетапну математичну задачу, показала критичну деградацію української мови з вигаданими словами та втратила числові дані при стисненні. На противагу їй, міркувальна qwen3:4b (10 балів) та хмарна модель (11 балів) бездоганно розв'язали логічну задачу з покроковим поясненням і продемонстрували чисту наукову термінологію. Водночас експеримент виявив спільну вразливість: попри здатність видавати чистий JSON у завданні 2, під час генерації коду моделі схильні порушувати негативну інструкцію «лише код», додаючи супровідний текст та markdown-розмітку через стійкі шаблони навчання.

Практичний висновок: оцінювання за рубрикою є незамінним фільтром перед вибором LLM для прикладних систем. Для задач логічного виведення та обробки текстів українською мовою надмалі локальні моделі класу 3B є непридатними,

тому пріоритет слід надавати моделям із вбудованим ланцюжком міркувань або потужним хмарним рішенням.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	5
temperature	1.5	5	5 (порушено формат)
top_p	0.1	1	5
top_p	0.5	5	5
top_p	1.0	5	5

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь:

Для повернення JSON слід встановлювати temperature = 0.0. Це мінімізує ентропію вибірки, виключає варіативність у синтаксичних символах і гарантує суворе дотримання схеми даних.

2. Яке — генератору варіантів заголовків?

Відповідь:

Оптимальним є діапазон $\text{temperature} = 0.7 - 0.8$ (або $\text{top_p} = 0.5-0.7$ при помірній температурі). Як видно з тесту, при $t=0.7$ модель дає 5 різних осмислених назв без переходу у вигадані слова.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь:

Temperature деформує математичний розподіл логітів: при $t=1.5$ модель підтягує хвіст рідкісних токенів і починає генерувати неіснуючі неологізми («Невистимирна», «*فیکس*»). top_p просто динамічно зрізає найменш імовірні токени, тому навіть при розширенні пулу слів текст залишається граматично притомнішим.

4. Чи означає $\text{temperature}=0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь:

Експеримент дав 10 збігів із 10 (100% однакових відповідей '2007.'). Проте в розподілених та високонавантажених системах абсолютна відтворюваність не гарантується через недетермінованість паралельних матричних операцій із рухомою комою в CUDA/cuBLAS.

5. Наведіть найбезглуздішу відповідь, яку модель видала при $\text{temperature}=1.5$, і поясніть, чому вона взагалі могла з'явитися.

Відповідь:

Найбільш дефектні варіанти: «*فیکس*» (з латинською F та суфіксом дієслова), «*Фільми на поромілю*» та «*Невистимирна зала*». При $t = 1.5$ розподіл імовірностей практично вирівнюється, через що модель починає обирати токени з мізерною вагою з іншомовних або рідкісних контекстів і «зліплює» вигадані слова.

Відповіді на 5 питання чат-боту Gemini (крок 8)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь:

Перед застосуванням функції Softmax логіти z_i діляться на значення температури T : $P(w_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$. При $T \rightarrow 0$ різниця між максимальним і рештою логітів експоненційно зростає, наближаючи вибір до $\arg\max$ (Greedy Search); при $T > 1$ логіти згладжуються, наближаючи розподіл до рівномірного шуму.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

Відповідь:

`Temperature` змінює форму та ентропію всього розподілу ймовірностей. `top_p` (Nucleus Sampling) динамічно обмежує вибірку мінімальним набором токенів, сума ймовірностей яких досягає порогу p , відсікаючи низькоймовірний шум, але не чіпаючи відносну вагу дозволених кандидатів.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь:

У більшості інженерних сценаріїв (зокрема в офіційній документації OpenAI) рекомендується змінювати лише один із них, фіксуючи інший на дефолтному значенні. Одночасна зміна ускладнює контроль над генерацією, хоча поєднання помірного $T \approx 0.7$ з $\text{top}_p \approx 0.9$ є поширеною практикою для фільтрації смітєвих токенів у творчих задачах.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь:

Сучасні фреймворки виконують матричні множення паралельно на тисячах ядер GPU або NPU. Через порядок зведення суми чисел з рухомою комою (float16/bfloat16) виникають мікроскопічні похибки округлення, яких іноді достатньо, щоб поміняти місцями два токени з практично однаковими логітами.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь:

Для RAG, витягу сутностей, кодогенерації та JSON-схем встановлюють $T = 0.0$ (максимум 0.2). Для копірайтингу, брейнштормінгу чи рольових діалогів задають $T = 0.7 - 0.9$.

Отже, для систем на основі виклику інструментів (AI-агенти, structured output) стандартні налаштування (temperature = 0.7–1.0) непридатні, оскільки вони провокують випадкові синтаксичні дефекти (зайві коми, невалідні лапки, галюцинації в ключах JSON). У подальших лабораторних роботах для роботи зі структурами даних слід безпеліційно виставляти temperature = 0.0 і звужувати top_p для максимальної стабільності парсингу.

Висновок за завданням 6

Експериментальне дослідження підтвердило ключову роль гіперпараметрів вибірки (temperature та top_p) у регулюванні стохастичності та ентропії відповідей мовної моделі. При фіксації temperature = 0.0 досягнуто повної детермінованості (1 унікальна відповідь із 5 у творчому та 100% повторюваності при 10 прогонах фактичного тесту), тоді як підвищення значення до 1.5 призвело до деградації лексичного апарату моделі через вирівнювання розподілу ймовірностей логітів Softmax і вибір токенів із маргінального хвоста (поява псевдослів на кшталт «Невистимирна» та «Фіксاتی»). Параметр top_p продемонстрував м'якший характер фільтрації, динамічно відсікаючи низькоймовірні токени за кумулятивним порогом без деформації ваги найбільш релевантних кандидатів.

Практичний висновок: для агентних систем, парсингу JSON та виклику інструментів критично фіксувати temperature = 0.0 для запобігання випадковим синтаксичним збоям у структурованих даних, тоді як помірний діапазон temperature = 0.7–0.8 (або top_p = 0.5–0.7) є доцільним лише для генеративних і евристичних завдань (брейнштормінг, підбір назв), де вимагається лексична варіативність без втрати змістовної зв'язності.

Підсумкова таблиця (Крок 8)

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Середня / низька на довгих текстах	Помірна	Висока
TTFT	Помірний / повільний	Повільний (генерація першого видимого токена затримується через попереднє формування блоку	Мінімальний
Якість, балів /12	4	10	11
Стабільність формату (завдання 2)	Висока	Висока	Висока
Потрібні ресурси	Мінімум RAM/VRAM (~2–3 ГБ)	4–6 ГБ VRAM, вимагає сучасного GPU для комфортної швидкості роздумів	Нульові локальні обчислювальні ресурси; потрібне лише стабільне інтернет-з'єднання
Вартість	Безкоштовно	Безкоштовно	Безкоштовні ліміти. У комерційному використанні — плата за 1M токенів
Куди йдуть ваші дані	Залишаються на локальній машині	Залишаються на локальній машині	Передаються на сторонні сервери провайдера
Працює без інтернету	Так, повністю автономно	Так, повністю автономно	Ні, вимагає стабільного інтернет-каналу
Коли обирати саме її	Коли потрібна 100% конфіденційність або офлайн-робота на слабкому залізі для найпростіших	Для локальних систем, де необхідні логічне/математичне мислення, багатокроковий	Для інтерактивних застосунків реального часу (завдяки низькому TTFT), пакетного аналізу довгих

	задач: класифікація, витяг сутностей за жорстким шаблоном, маршрутизація промптів користувача. Категорично не підходить для складних розрахунків та українських текстів.	аналіз та висока грамотність української мови, за умови наявності дискретної GPU і готовності чекати генерацію прихованого ланцюжка думок.	документів, написання складного коду та високих вимог до мовної культури, якщо оброблювані дані не є банківською/державною таємницею чи суворим NDA.
--	--	--	--

Загальні висновки

У результаті виконання лабораторної роботи було розгорнуто локальне середовище інференсу на базі сервісу Ollama та практично доведено переваги стандартизації через OpenAI-сумісний інтерфейс. Перемикання між локальними моделями та хмарними платформами здійснюється виключно через зміну параметрів конфігурації без модифікації коду програми, що підтверджує критичну важливість такої архітектурної абстракції для безшовного резервування провайдерів у прикладних системах.

Експериментальні заміри продуктивності засвідчили, що загальну затримку неможливо оцінити одним числом, оскільки вона формується двома незалежними показниками — часом до появи першого токена (TTFT) та швидкістю подальшого декодування. Хмарна інфраструктура Groq продемонструвала багаторазову перевагу над локальною системою саме завдяки скороченню TTFT до ~0,5 с та високій швидкості генерації на довгих текстах (до 1982 симв./с), водночас аналіз локального запуску виявив дворазове сповільнення первинного відгуку при холодному старті через операції завантаження вагів із диска.

Порівняння міркувальної архітектури qwen3:4b з інструктивною llama3.2:3b визначило чіткі межі їх ефективності. Внутрішній ланцюжок думок виявився вирішальним для розв'язання складних логічних і математичних задач, забезпечивши стовідсоткову точність проти 40% в інструктивній моделі, проте на простих фактах він став причиною колосальної перевитрати токенів (у 71 раз) і часу очікування (у 81 раз). Це формує інженерне правило: міркувальні моделі мають застосовуватися суворо як цільові аналітичні вузли, а не для типових потокових запитів.

Оцінювання якості за фіксованою рубрикою підтвердило непридатність надмалих моделей класу 3B для складного аналізу українською мовою через критичну втрату числових даних, помилкові обчислення та появу неологізмів. Разом із цим було наочно продемонстровано вразливість чисто текстових інструкцій: навіть великі моделі періодично порушують обмеження формату або негативні вказівки (наприклад, додають markdown до коду), що робить обов'язковим використання програмної валідації схем (JSON Schema / Pydantic) на рівні бекенду замість сліпої довіри промπτу.

Дослідження стохастичності показало, що LLM є ймовірнісними системами, де фіксація параметра `temperature = 0.0` є безальтернативною для отримання стабільних машинозчитуваних відповідей, тоді як підвищення температури до 1.5 повністю руйнує синтаксичну та семантичну цілісність тексту. Будь-яке об'єктивне вимірювання чи бенчмаркінг моделей вимагає серії повторів із вибором медіани, що нівелює системний шум паралельних обчислень і гарантує відтворюваність інженерних рішень.